

Python Interview & Coding Study Guide

Interview Preparation • Coding Challenges • LeetCode & HackerRank

Table of Contents

- [Data Types & Variables](#1-data-types--variables)
 - [Strings & String Methods](#2-strings--string-methods)
 - [Lists & List Comprehensions](#3-lists--list-comprehensions)
 - [Tuples & Sets](#4-tuples--sets)
 - [Dictionaries](#5-dictionaries)
 - [Control Flow](#6-control-flow)
 - [Functions & Scope](#7-functions--scope)
 - [Object-Oriented Programming](#8-object-oriented-programming)
 - [Iterators & Generators](#9-iterators--generators)
 - [Error Handling](#10-error-handling)
 - [File Handling](#11-file-handling)
 - [Sorting & Searching Algorithms](#12-sorting--searching-algorithms)
 - [Recursion](#13-recursion)
 - [Dynamic Programming](#14-dynamic-programming)
 - [Common Interview Patterns](#15-common-interview-patterns)
-

1. Data Types & Variables

Concept Overview

Python has several built-in data types: integers (`int`), floating-point numbers (`float`), booleans (`bool`), and `None`. Understanding type coercion, mutability, and identity vs. equality is essential for interviews.

Example 1: Integer, Float, and Boolean Basics

```
# Basic numeric types and operations
x = 10          # int
y = 3.14        # float
flag = True     # bool

print(type(x))    # Output: <class 'int'>
print(type(y))    # Output: <class 'float'>
print(x + y)      # Output: 13.14
print(int(y))     # Output: 3 (truncates, does not round)
print(bool(0))    # Output: False
print(bool(""))   # Output: False (empty string is falsy)
print(bool([]))   # Output: False (empty list is falsy)
```

Example 2: Identity vs. Equality

```
# == checks value equality; 'is' checks object identity
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)    # Output: True (same values)
print(a is b)    # Output: False (different objects in memory)
print(a is c)    # Output: True (same object reference)

# Integer caching: Python caches small integers (-5 to 256)
x = 256
y = 256
print(x is y)    # Output: True (cached)

x = 257
y = 257
print(x is y)    # Output: False (not cached – implementation detail)
```

Example 3: Type Conversion & Common Pitfalls

```
# Implicit vs. explicit conversion
print(5 / 2)     # Output: 2.5 (true division in Python 3)
print(5 // 2)    # Output: 2 (floor division)
print(5 % 2)     # Output: 1 (modulo)
print(2 ** 10)   # Output: 1024 (exponentiation)

# Careful with float precision
print(0.1 + 0.2) # Output: 0.30000000000000004
print(0.1 + 0.2 == 0.3) # Output: False (floating-point pitfall!)
import math
print(math.isclose(0.1 + 0.2, 0.3)) # Output: True (correct comparison)
```

Example 4: Multiple Assignment & Swapping

```
# Python allows elegant multiple assignment
a, b, c = 1, 2, 3
print(a, b, c) # Output: 1 2 3

# Swap without a temp variable (interview classic)
a, b = b, a
print(a, b)    # Output: 2 1

# Unpacking with star
first, *rest = [1, 2, 3, 4, 5]
print(first)   # Output: 1
print(rest)    # Output: [2, 3, 4, 5]
```

Interview Tips

- **Falsy values:** 0, 0.0, "", [], {}, (), None, and False are all falsy.
- **is vs ==:** Always use == for value comparison; use is only when checking for None (e.g., if x is None:).
- **Common Question:** "What is the difference between // and /?" — Floor division vs. true division.
- **Edge Case:** Integer overflow doesn't exist in Python 3; integers grow arbitrarily large.

2. Strings & String Methods

Concept Overview

Strings are immutable sequences of characters. Python's rich string methods and f-strings make string manipulation elegant. String problems are among the most common interview topics.

Example 1: String Slicing & Indexing

```
s = "Hello, World!"

# Indexing (0-based, negative from end)
print(s[0])      # Output: H
print(s[-1])     # Output: !
print(s[7:12])   # Output: World
print(s[:5])     # Output: Hello
print(s[::2])    # Output: Hlo ol! (every second character)
print(s[::-1])   # Output: !dlrow ,olleH (reverse string)
```

Example 2: Common String Methods

```

text = " Python is Amazing "

print(text.strip())          # Output: Python is Amazing
print(text.lower())          # Output: python is amazing
print(text.upper())          # Output: PYTHON IS AMAZING
print(text.replace("Amazing", "Great")) # Output: Python is Great
print("python" in text.lower()) # Output: True

words = text.strip().split(" ")
print(words)                 # Output: ['Python', 'is', 'Amazing']
print(" ".join(words))       # Output: Python is Amazing
print(text.strip().count("i")) # Output: 2

```

Example 3: F-Strings & Formatting

```

name = "Alice"
score = 95.678

# f-string (Python 3.6+)
print(f"Name: {name}, Score: {score:.2f}")
# Output: Name: Alice, Score: 95.68

# Useful in interviews for clean output
items = [1, 2, 3]
print(f"Length: {len(items)}, Sum: {sum(items)}")
# Output: Length: 3, Sum: 6

# Padding and alignment
for i in range(1, 4):
    print(f"{i:>3}: {'*' * i}")
# Output:
# 1: *
# 2: **
# 3: ***

```

Example 4: Anagram & Palindrome Check

```
# Classic interview problems
def is_anagram(s1, s2):
    """Check if two strings are anagrams. O(n) time, O(1) space (26 chars)."""
    return sorted(s1.lower()) == sorted(s2.lower())

def is_palindrome(s):
    """Check palindrome ignoring spaces and case."""
    cleaned = ''.join(c.lower() for c in s if c.isalnum())
    return cleaned == cleaned[::-1]

print(is_anagram("listen", "silent")) # Output: True
print(is_anagram("hello", "world"))  # Output: False

print(is_palindrome("A man a plan a canal Panama")) # Output: True
print(is_palindrome("race a car"))                  # Output: False
```

Example 5: String Compression (Interview Question)

```
def compress_string(s):
    """Compress 'aaabbc' to 'a3b2c1'. Return original if compressed is longer."""
    if not s:
        return s

    compressed = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            compressed.append(s[i - 1] + str(count))
            count = 1
    compressed.append(s[-1] + str(count))

    result = ''.join(compressed)
    return result if len(result) < len(s) else s

print(compress_string("aaabbc")) # Output: a3b2c1
print(compress_string("abcd"))   # Output: abcd (no compression benefit)
```

Interview Tips

- **Immutability:** Use `"".join(list)` to build strings in loops — $O(n)$, not $O(n^2)$ like `+=`.
- `ord()` / `chr()`: Convert character to ASCII and back; useful for cipher problems.
- **Common Question:** "Find the first non-repeating character in a string."
- **Edge Cases:** Empty strings, strings with only spaces, Unicode characters.

3. Lists & List Comprehensions

Concept Overview

Lists are mutable, ordered, zero-indexed sequences. They support $O(1)$ append/pop from the end and $O(n)$ insert/delete elsewhere. List comprehensions are a Pythonic, efficient way to create lists.

Example 1: Core List Operations

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

nums.append(7)      # Add to end: O(1)
nums.insert(0, 0)   # Insert at index: O(n)
nums.pop()          # Remove last: O(1)
nums.pop(0)         # Remove at index: O(n)
nums.remove(1)      # Remove first occurrence: O(n)

print(nums)         # Output: [1, 4, 5, 9, 2, 6]
print(len(nums))    # Output: 6
print(max(nums))    # Output: 9
print(min(nums))    # Output: 1
print(sum(nums))    # Output: 27
print(nums.index(9)) # Output: 3 (first position of 9)
print(nums.count(1)) # Output: 1
```

Example 2: List Comprehensions — Basic to Advanced

```
# Basic: squares
squares = [x**2 for x in range(10)]
print(squares) # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With condition: even numbers only
evens = [x for x in range(20) if x % 2 == 0]
print(evens)   # Output: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Nested: flatten a 2D list
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
print(flat)   # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Conditional expression (ternary) in comprehension
result = ["even" if x % 2 == 0 else "odd" for x in range(6)]
print(result) # Output: ['even', 'odd', 'even', 'odd', 'even', 'odd']
```

Example 3: Two-Pointer Technique on Lists

```
def two_sum_sorted(nums, target):
    """
    Find two indices in a SORTED list that sum to target.
    Time: O(n), Space: O(1)
    """
    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return (left, right)
        elif current_sum < target:
            left += 1
        else:
            right -= 1
    return None

nums = [2, 7, 11, 15]
print(two_sum_sorted(nums, 9))    # Output: (0, 1)
print(two_sum_sorted(nums, 18))   # Output: (1, 2)
```

Example 4: Sliding Window on a List

```
def max_subarray_sum(nums, k):
    """
    Maximum sum of a subarray of length k.
    Time: O(n), Space: O(1)
    """
    if len(nums) < k:
        return None

    window_sum = sum(nums[:k])
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum

print(max_subarray_sum([2, 1, 5, 1, 3, 2], 3)) # Output: 9 (5+1+3)
print(max_subarray_sum([2, 3, 4, 1, 5], 2))    # Output: 7 (3+4)
```

Interview Tips

- **Complexity:** Append/pop from end is $O(1)$; insert/remove at index is $O(n)$.
- **Deep vs. Shallow Copy:** `b = a` is a reference; `b = a[:]` or `b = a.copy()` is a shallow copy; `copy.deepcopy(a)` for nested structures.
- **Common Question:** "Find the largest subarray with sum equal to k" (use prefix sums + hash map).
- **Pitfall:** Never modify a list while iterating over it — iterate over a copy instead.

4. Tuples & Sets

Concept Overview

Tuples are immutable sequences, ideal for fixed data and dictionary keys. **Sets** are unordered collections of unique elements supporting O(1) average-case lookups, useful for membership testing and deduplication.

Example 1: Tuple Basics and Unpacking

```
# Tuples: immutable, hashable (can be dict keys or set elements)
point = (3, 4)
x, y = point # unpacking
print(x, y)  # Output: 3 4

# Named tuple for readable code
from collections import namedtuple
Person = namedtuple('Person', ['name', 'age'])
alice = Person(name="Alice", age=30)
print(alice.name, alice.age) # Output: Alice 30
print(alice[0])              # Output: Alice (index access also works)

# Tuples as dictionary keys (lists cannot be dict keys)
distances = {(0, 0): 0, (1, 2): 5, (3, 4): 7}
print(distances[(1, 2)]) # Output: 5
```

Example 2: Set Operations — Interview Essentials

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

print(a | b) # Union: {1, 2, 3, 4, 5, 6, 7, 8}
print(a & b) # Intersection: {4, 5}
print(a - b) # Difference: {1, 2, 3}
print(a ^ b) # Symmetric diff: {1, 2, 3, 6, 7, 8}

# Membership testing: O(1) average
print(3 in a) # Output: True
print(9 in a) # Output: False

# Deduplication
nums = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(nums))
print(sorted(unique)) # Output: [1, 2, 3, 4]
```

Example 3: Finding Duplicates with Sets


```
def contains_duplicate(nums):
    """O(n) time, O(n) space."""
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False

def find_all_duplicates(nums):
    """Return all elements appearing more than once."""
    seen = set()
    duplicates = set()
    for num in nums:
        if num in seen:
            duplicates.add(num)
        seen.add(num)
    return list(duplicates)

print(contains_duplicate([1, 2, 3, 4]))    # Output: False
print(contains_duplicate([1, 2, 3, 1]))    # Output: True
print(find_all_duplicates([1, 2, 2, 3, 3, 4])) # Output: [2, 3]
```

Interview Tips

- **Frozenset:** An immutable set; can be used as dictionary keys or in sets of sets.
- **Common Question:** "Given two arrays, find their intersection" — use set intersection.
- **Pitfall:** Sets are unordered; do not assume iteration order.
- **Tuple vs List:** Prefer tuples for heterogeneous fixed-length data; lists for homogeneous variable-length data.

5. Dictionaries

Concept Overview

Dictionaries are hash maps with $O(1)$ average-case lookup, insertion, and deletion. In Python 3.7+, they maintain insertion order. They are the foundation of many efficient algorithmic solutions.

Example 1: Core Dictionary Operations

```

# Create, access, update, delete
student = {"name": "Alice", "grade": 90, "subject": "Math"}

print(student["name"])          # Output: Alice
print(student.get("age", "N/A")) # Output: N/A (safe access with default)

student["grade"] = 95          # Update
student["school"] = "MIT"      # Insert new key

del student["subject"]         # Delete key
popped = student.pop("school")  # Remove & return value
print(popped)                  # Output: MIT

# Iteration
for key, value in student.items():
    print(f"{key}: {value}")
# Output:
# name: Alice
# grade: 95

```

Example 2: defaultdict & Counter

```

from collections import defaultdict, Counter

# defaultdict: auto-initializes missing keys
graph = defaultdict(list)
graph["A"].append("B")
graph["A"].append("C")
graph["B"].append("D")
print(dict(graph)) # Output: {'A': ['B', 'C'], 'B': ['D']}

# Counter: count frequencies efficiently
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
count = Counter(words)
print(count)          # Output: Counter({'apple': 3, 'banana': 2, 'cherry': 1})
print(count.most_common(2)) # Output: [('apple', 3), ('banana', 2)]
print(count["apple"])    # Output: 3

```

Example 3: Two-Sum with Hash Map (Classic Interview)

```
def two_sum(nums, target):
    """
    Return indices of two numbers that sum to target.
    Time: O(n), Space: O(n)
    """
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
print(two_sum([3, 2, 4], 6))    # Output: [1, 2]
print(two_sum([3, 3], 6))       # Output: [0, 1]
```

Example 4: Group Anagrams

```
from collections import defaultdict

def group_anagrams(strs):
    """
    Group strings that are anagrams of each other.
    Time: O(n * k log k), Space: O(n * k) where k = max string length
    """
    groups = defaultdict(list)
    for s in strs:
        key = tuple(sorted(s)) # Sorted characters as key
        groups[key].append(s)
    return list(groups.values())

words = ["eat", "tea", "tan", "ate", "nat", "bat"]
print(group_anagrams(words))
# Output: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Example 5: LRU Cache (Advanced)

```

from collections import OrderedDict

class LRUCache:
    """
    Least Recently Used Cache using OrderedDict.
    O(1) get and put operations.
    """
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key) # Mark as recently used
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False) # Remove least recently used

cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
print(cache.get(1)) # Output: 1
cache.put(3, 3)     # Evicts key 2
print(cache.get(2)) # Output: -1 (evicted)

```

Interview Tips

- **O(1) Operations:** Lookup, insert, and delete are O(1) average, O(n) worst case (hash collision).
- **get() vs []:** Use `.get(key, default)` to avoid `KeyError`.
- **Common Question:** "Find the first non-repeating character" — use a dict to count, then scan.
- **Pitfall:** Dictionary keys must be hashable (immutable). Lists cannot be keys; tuples can.

6. Control Flow

Concept Overview

Python control flow constructs include `if/elif/else`, `for`, `while`, `break`, `continue`, and `pass`. Understanding loop optimization and short-circuit evaluation is crucial.

Example 1: Conditional Expressions & Short-Circuit

```

# Short-circuit evaluation
def expensive():
    print("Called!")
    return True

x = False
result = x and expensive() # expensive() is NOT called (short-circuit)
print(result) # Output: False

y = True
result = y or expensive() # expensive() is NOT called (short-circuit)
print(result) # Output: True

# Ternary operator
age = 20
status = "adult" if age >= 18 else "minor"
print(status) # Output: adult

```

Example 2: For Loop Patterns

```

fruits = ["apple", "banana", "cherry"]

# enumerate: index + value
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# Output: 0: apple 1: banana 2: cherry

# zip: iterate two lists together
prices = [1.5, 2.0, 3.5]
for fruit, price in zip(fruits, prices):
    print(f"{fruit}: ${price}")
# Output: apple: $1.5 banana: $2.0 cherry: $3.5

# range: start, stop, step
for i in range(10, 0, -2):
    print(i, end=" ")
print() # Output: 10 8 6 4 2

```

Example 3: While Loop & Break/Continue

```
# Binary search using while loop
def binary_search(arr, target):
    """Time: O(log n), Space: O(1)"""
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2 # Avoids potential overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

arr = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(arr, 7)) # Output: 3
print(binary_search(arr, 4)) # Output: -1
```

Example 4: For/Else and While/Else

```
# Python's unique for/else: else runs if loop didn't 'break'
def find_prime(n):
    """Check if n is prime using for/else."""
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False # break would skip the else
    else:
        return True # Only runs if no break occurred

for num in [2, 3, 4, 17, 20]:
    print(f"{num} is {'prime' if find_prime(num) else 'not prime'}")
# Output: 2 is prime, 3 is prime, 4 is not prime, 17 is prime, 20 is not prime
```

Interview Tips

- `enumerate()`: Always use instead of `for i in range(len(arr))` — more Pythonic.
- `zip()`: Stops at the shortest iterable; use `itertools.zip_longest` to pad.
- `for/else`: Unique to Python — extremely useful for search loops.
- **Common Question**: "Write a loop to check if an array is sorted."

7. Functions & Scope

Concept Overview

Functions are first-class objects in Python. Understanding closures, decorators, `*args/**kwargs`, and scope (LEGB rule) is vital for both interviews and writing clean code.

Example 1: `*args` and `**kwargs`

```
def stats(*args, **kwargs):
    """Accept any number of positional and keyword arguments."""
    label = kwargs.get("label", "Values")
    if args:
        print(f"{label}: {args}")
        print(f"  Sum: {sum(args)}, Mean: {sum(args)/len(args):.2f}")

stats(1, 2, 3, 4, 5, label="Numbers")
# Output:
# Numbers: (1, 2, 3, 4, 5)
#   Sum: 15, Mean: 3.00

# Unpacking into a function
nums = [1, 2, 3]
print(max(*nums)) # Output: 3 – equivalent to max(1, 2, 3)
```

Example 2: `Lambda`, `map`, `filter`, `sorted`

```
# Lambda: anonymous single-expression functions
square = lambda x: x ** 2
print(square(5)) # Output: 25

# map: apply function to all elements
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, nums))
print(squares) # Output: [1, 4, 9, 16, 25]

# filter: keep elements that satisfy condition
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # Output: [2, 4]

# sorted with key function
words = ["banana", "apple", "cherry", "date"]
print(sorted(words, key=len)) # Output: ['date', 'apple', 'banana', 'cherry']
print(sorted(words, key=len, reverse=True)) # Output: ['banana', 'cherry', 'apple', 'date']

# Sort tuples by second element
pairs = [(1, 3), (2, 1), (3, 2)]
print(sorted(pairs, key=lambda x: x[1])) # Output: [(2, 1), (3, 2), (1, 3)]
```

Example 3: `Closures`

```
def make_counter(start=0):
    """Returns a counter function with its own state."""
    count = [start] # Use list to allow mutation in closure
    def counter():
        count[0] += 1
        return count[0]
    return counter

c1 = make_counter()
c2 = make_counter(10)
print(c1()) # Output: 1
print(c1()) # Output: 2
print(c2()) # Output: 11
print(c1()) # Output: 3 (independent from c2)
```

Example 4: Decorators

```
import time
import functools

def timer(func):
    """Decorator that measures function execution time."""
    @functools.wraps(func) # Preserves original function metadata
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

result = slow_sum(1_000_000)
print(result)
# Output: slow_sum took 0.0234s
#         499999500000
```

Example 5: Memoization with `functools.lru_cache`


```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    """Memoized Fibonacci – O(n) time with @lru_cache."""
    if n <= 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(50)) # Output: 12586269025
print(fibonacci.cache_info()) # CacheInfo(hits=48, misses=51, ...)
```

Interview Tips

- **LEGB Rule:** Local → Enclosing → Global → Built-in; Python resolves names in this order.
- `nonlocal`: Use in nested functions to modify enclosing scope variables.
- `functools.wraps`: Always use when writing decorators to preserve `__name__` and `__doc__`.
- **Common Question:** "Implement a memoize decorator from scratch."

8. Object-Oriented Programming

Concept Overview

OOP in Python involves classes, objects, inheritance, encapsulation, and polymorphism. Understanding dunder methods (`__init__`, `__repr__`, etc.) and class vs. instance attributes is critical.

Example 1: Class Basics with Dunder Methods

```

class Stack:
    """LIFO data structure implemented with a list."""

    def __init__(self):
        self._data = [] # Single underscore: internal use convention

    def push(self, item):
        self._data.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self._data.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek at empty stack")
        return self._data[-1]

    def is_empty(self):
        return len(self._data) == 0

    def __len__(self):
        return len(self._data)

    def __repr__(self):
        return f"Stack({self._data})"

s = Stack()
s.push(1)
s.push(2)
s.push(3)
print(s)          # Output: Stack([1, 2, 3])
print(len(s))     # Output: 3
print(s.pop())    # Output: 3
print(s.peek())   # Output: 2

```

Example 2: Inheritance and Polymorphism

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclass must implement speak()")

    def __repr__(self):
        return f"{self.__class__.__name__}(name={self.name!r})"

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [Dog("Rex"), Cat("Whiskers"), Dog("Buddy")]
for animal in animals:
    print(animal.speak())
# Output: Rex says Woof! Whiskers says Meow! Buddy says Woof!

# isinstance and issubclass
print(isinstance(animals[0], Animal)) # Output: True
print(issubclass(Dog, Animal))        # Output: True

```

Example 3: Class Methods and Static Methods

```

class Temperature:
    absolute_zero = -273.15 # Class variable

    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    @classmethod
    def from_fahrenheit(cls, f):
        """Alternative constructor."""
        return cls((f - 32) * 5 / 9)

    @staticmethod
    def is_valid(celsius):
        return celsius >= Temperature.absolute_zero

    def __repr__(self):
        return f"Temperature({self.celsius}°C)"

t = Temperature(100)
print(t.fahrenheit)                # Output: 212.0

t2 = Temperature.from_fahrenheit(32)
print(t2)                          # Output: Temperature(0.0°C)

print(Temperature.is_valid(-300))   # Output: False
print(Temperature.is_valid(25))    # Output: True

```

Example 4: `__eq__`, `__hash__`, and `__lt__` (Comparisons)

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return isinstance(other, Point) and self.x == other.x and self.y == other.y

    def __hash__(self):
        return hash((self.x, self.y)) # Must define __hash__ if __eq__ is defined

    def __lt__(self, other):
        return (self.x**2 + self.y**2) < (other.x**2 + other.y**2)

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)

print(p1 == p2)      # Output: True
print(p1 is p2)      # Output: False
print(p1 < p3)       # Output: True (distance from origin)
print(sorted([p3, p1, p2])) # Output: [Point(1, 2), Point(1, 2), Point(3, 4)]

# Can now use in sets and as dict keys
points = {p1, p2, p3}
print(len(points))   # Output: 2 (p1 and p2 are equal)

```

Interview Tips

- **__repr__ vs __str__:** __repr__ for developers (unambiguous); __str__ for end users.
- **MRO:** Python uses C3 linearization for multiple inheritance. `ClassName.__mro__` shows the order.
- **Abstract classes:** Use from `abc import ABC, abstractmethod` to enforce interface contracts.
- **Common Question:** "Design a deck of cards class hierarchy."

9. Iterators & Generators

Concept Overview

Iterators implement `__iter__` and `__next__`. Generators use `yield` to lazily produce values, offering huge memory savings for large datasets. These are common in both interviews and production code.

Example 1: Custom Iterator

```

class Countdown:
    """Iterator that counts down from n to 0."""
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        val = self.n
        self.n -= 1
        return val

for num in Countdown(5):
    print(num, end=" ")
# Output: 5 4 3 2 1 0

```

Example 2: Generator Functions

```

def fibonacci_gen(limit):
    """Lazily generate Fibonacci numbers up to limit."""
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b

# Memory-efficient: only one value in memory at a time
for fib in fibonacci_gen(100):
    print(fib, end=" ")
# Output: 0 1 1 2 3 5 8 13 21 34 55 89

# Generator expression (like list comprehension but lazy)
gen_expr = (x**2 for x in range(1_000_000)) # No memory allocated yet
print(next(gen_expr)) # Output: 0
print(next(gen_expr)) # Output: 1

```

Example 3: Generator Pipeline (Real-World)

```
def read_large_file(filename):
    """Read a file line by line (simulated here)."""
    lines = ["10\n", "abc\n", "20\n", "30\n", "xyz\n", "40\n"]
    for line in lines:
        yield line.strip()

def parse_numbers(lines):
    """Filter and parse only numeric lines."""
    for line in lines:
        if line.isdigit():
            yield int(line)

def multiply_by(values, factor):
    """Multiply each value by a factor."""
    for v in values:
        yield v * factor

# Compose the pipeline (no large lists in memory)
lines = read_large_file("data.txt")
numbers = parse_numbers(lines)
doubled = multiply_by(numbers, 2)

print(list(doubled)) # Output: [20, 40, 60, 80]
```

Interview Tips

- **Memory Efficiency:** Generators are $O(1)$ memory regardless of data size.
- **yield from:** Delegate to a sub-generator cleanly.
- **Common Question:** "Implement an infinite sequence generator."
- **Pitfall:** A generator can only be iterated once; after exhaustion, it returns nothing.

10. Error Handling

Concept Overview

Python uses `try/except/else/finally` for exception handling. Defining custom exceptions and understanding the exception hierarchy are important for writing robust code.

Example 1: Try/Except/Else/Finally

```
def safe_divide(a, b):
    """Demonstrate full try/except/else/finally pattern."""
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError as e:
        print(f"Error: Invalid types – {e}")
        return None
    else:
        # Runs only if no exception occurred
        print(f"Success: {a} / {b} = {result:.4f}")
        return result
    finally:
        # ALWAYS runs (cleanup)
        print("Division operation complete.")

safe_divide(10, 2) # Success: 10 / 2 = 5.0000 | Division operation complete.
safe_divide(10, 0) # Error: Cannot divide by zero! | Division operation complete.
safe_divide(10, "a") # Error: Invalid types | Division operation complete.
```

Example 2: Custom Exceptions

```
class ValidationError(Exception):
    """Raised when input data fails validation."""
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(f"Validation failed on '{field}': {message}")

class AgeError(ValidationError):
    """Specific validation error for age."""
    pass

def validate_user(name, age):
    if not name or not name.strip():
        raise ValidationError("name", "Name cannot be empty")
    if not isinstance(age, int) or age < 0 or age > 150:
        raise AgeError("age", f"Age {age} is out of valid range (0-150)")
    return {"name": name, "age": age}

try:
    user = validate_user("", 25)
except AgeError as e:
    print(f"Age error: {e}")
except ValidationError as e:
    print(f"Validation error: {e}") # Output: Validation error: Validation failed on 'name': Na
```

Example 3: Context Managers with with


```

class DatabaseConnection:
    """Simulated database connection using context manager protocol."""
    def __init__(self, db_name):
        self.db_name = db_name
        self.connected = False

    def __enter__(self):
        print(f"Connecting to {self.db_name}...")
        self.connected = True
        return self # Value bound to 'as' variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Disconnecting from {self.db_name}...")
        self.connected = False
        return False # Re-raise any exception (True would suppress it)

    def query(self, sql):
        if not self.connected:
            raise RuntimeError("Not connected!")
        return f"Results of: {sql}"

with DatabaseConnection("mydb") as db:
    print(db.query("SELECT * FROM users"))
# Output:
# Connecting to mydb...
# Results of: SELECT * FROM users
# Disconnecting from mydb...

```

Interview Tips

- **Exception Hierarchy:** All exceptions inherit from `BaseException`; most from `Exception`. Use `Exception` as base for custom exceptions.
- **else clause:** Only runs if no exception was raised — great for code that should run on success.
- **finally:** Always executes — use for cleanup (closing files, releasing locks).
- **Common Question:** "What's the difference between `except Exception` and `except BaseException`?" (SystemExit, KeyboardInterrupt inherit from `BaseException` only).

11. File Handling

Concept Overview

Python's `open()` and context managers make file I/O safe and clean. Understanding read modes, encoding, and working with JSON/CSV files is essential for practical coding.

Example 1: Reading and Writing Text Files

```

# Write to a file
with open("sample.txt", "w", encoding="utf-8") as f:
    f.write("Hello, World!\n")
    f.writelines(["Line 2\n", "Line 3\n"])

# Read entire file
with open("sample.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(repr(content))
# Output: 'Hello, World!\nLine 2\nLine 3\n'

# Read line by line (memory-efficient for large files)
with open("sample.txt", "r") as f:
    for i, line in enumerate(f, start=1):
        print(f"Line {i}: {line.strip()}")
# Output: Line 1: Hello, World!  Line 2: Line 2  Line 3: Line 3

```

Example 2: JSON File Handling

```

import json

# Write JSON
data = {
    "users": [
        {"id": 1, "name": "Alice", "scores": [90, 85, 92]},
        {"id": 2, "name": "Bob", "scores": [78, 82, 88]}
    ]
}

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

# Read JSON
with open("data.json", "r") as f:
    loaded = json.load(f)

for user in loaded["users"]:
    avg = sum(user["scores"]) / len(user["scores"])
    print(f"{user['name']}: avg score = {avg:.1f}")
# Output: Alice: avg score = 89.0  Bob: avg score = 82.7

```

Example 3: CSV and pathlib

```

import csv
from pathlib import Path

# Using pathlib for cleaner paths
data_dir = Path(".")
csv_file = data_dir / "students.csv"

# Write CSV
rows = [
    ["Name", "Grade", "Score"],
    ["Alice", "A", 95],
    ["Bob", "B", 82],
    ["Charlie", "A-", 90]]

with open(csv_file, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(rows)

# Read CSV as dictionaries
with open(csv_file, "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['Name']}: {row['Score']}")

# Output: Alice: 95 Bob: 82 Charlie: 90

```

Interview Tips

- **Always use with:** Ensures the file is closed even if an exception occurs.
- **Encoding:** Always specify `encoding="utf-8"` to avoid platform-dependent issues.
- `pathlib.Path`: Preferred over `os.path` for modern Python code.
- **Common Question:** "How would you count word frequencies in a large text file?"

12. Sorting & Searching Algorithms

Concept Overview

Python's built-in `sorted()` uses Timsort ($O(n \log n)$, stable). Knowing how to implement and analyze sorting and searching algorithms is critical for technical interviews.

Example 1: Merge Sort

```

def merge_sort(arr):
    """
    Divide-and-conquer sorting.
    Time: O(n log n), Space: O(n)
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(arr)) # Output: [3, 9, 10, 27, 38, 43, 82]

```

Example 2: Quick Sort

```

def quick_sort(arr, low=0, high=None):
    """
    In-place sorting using partitioning.
    Time: O(n log n) average, O(n^2) worst, Space: O(log n)
    """
    if high is None:
        high = len(arr) - 1
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

arr = [64, 25, 12, 22, 11]
quick_sort(arr)
print(arr) # Output: [11, 12, 22, 25, 64]

```

Example 3: Binary Search (Iterative & Recursive)

```

def binary_search_iterative(arr, target):
    """Time: O(log n), Space: O(1)"""
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def binary_search_recursive(arr, target, left=0, right=None):
    """Time: O(log n), Space: O(log n) due to call stack"""
    if right is None:
        right = len(arr) - 1
    if left > right:
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)

arr = [1, 3, 5, 7, 9, 11, 13, 15]
print(binary_search_iterative(arr, 7))    # Output: 3
print(binary_search_recursive(arr, 13))   # Output: 6
print(binary_search_iterative(arr, 4))    # Output: -1

```

Example 4: Using bisect for Sorted Lists

```

import bisect

sorted_arr = [1, 3, 4, 7, 10, 15]

# Insert while maintaining sort order
bisect.insort(sorted_arr, 5)
print(sorted_arr) # Output: [1, 3, 4, 5, 7, 10, 15]

# Find insertion point
idx = bisect.bisect_left(sorted_arr, 7)
print(idx)        # Output: 4 (index where 7 is)

# Count elements less than a value
def count_less_than(arr, target):
    return bisect.bisect_left(arr, target)

print(count_less_than(sorted_arr, 7)) # Output: 4 (elements: 1,3,4,5)

```

Interview Tips

- **Timsort:** Python's built-in sort is stable (preserves relative order of equal elements).
 - **Binary search on answer:** Many problems use binary search not on data, but on the answer space.
 - **Common Question:** "Find the kth largest element" — use `heapq.nlargest(k, arr)` or `quickselect`.
-

13. Recursion

Concept Overview

Recursion solves problems by breaking them into smaller subproblems. Every recursive function needs a base case and a recursive case. Understanding the call stack is key.

Example 1: Classic Recursion — Factorial and Power

```
def factorial(n):
    """n! — Time: O(n), Space: O(n) (call stack)"""
    if n == 0 or n == 1: # Base case
        return 1
    return n * factorial(n - 1) # Recursive case

def power(base, exp):
    """Fast exponentiation — Time: O(log n)"""
    if exp == 0:
        return 1
    if exp % 2 == 0:
        half = power(base, exp // 2)
        return half * half
    return base * power(base, exp - 1)

print(factorial(6))    # Output: 720
print(power(2, 10))   # Output: 1024
```

Example 2: Tree Traversal (Binary Tree)

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder(root):
    """Left → Root → Right – gives sorted order for BST"""
    if root is None:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def max_depth(root):
    """Maximum depth of binary tree. Time: O(n), Space: O(h)"""
    if root is None:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

# Build: 1 -> (2, 3), 2 -> (4, 5)
root = TreeNode(1,
    TreeNode(2, TreeNode(4), TreeNode(5)),
    TreeNode(3))

print(inorder(root))      # Output: [4, 2, 5, 1, 3]
print(max_depth(root))    # Output: 3

```

Example 3: Backtracking — Generate Permutations

```

def permutations(nums):
    """
    Generate all permutations using backtracking.
    Time: O(n * n!), Space: O(n)
    """
    result = []

    def backtrack(current, remaining):
        if not remaining:
            result.append(current[:]) # Append a copy
            return
        for i, num in enumerate(remaining):
            current.append(num)
            backtrack(current, remaining[:i] + remaining[i+1:])
            current.pop() # Undo the choice (backtrack)

    backtrack([], nums)
    return result

print(permutations([1, 2, 3]))
# Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

```


Interview Tips

- **Stack Depth:** Python's default recursion limit is 1000. Use `sys.setrecursionlimit()` or convert to iteration for deep recursions.
 - **Tail Recursion:** Python does not optimize tail calls; always consider iterative alternatives.
 - **Common Pattern:** Backtracking = "choose, explore, un-choose."
 - **Common Question:** "Solve the N-Queens problem" or "Generate all subsets."
-

14. Dynamic Programming

Concept Overview

DP solves problems by storing results of subproblems (memoization = top-down; tabulation = bottom-up). It applies when a problem has overlapping subproblems and optimal substructure.

Example 1: Fibonacci — Memoization vs. Tabulation

```

# Top-down: memoization (recursive + cache)
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

# Bottom-up: tabulation (iterative)
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Space-optimized bottom-up: O(1) space
def fib_opt(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

print(fib_memo(40))    # Output: 102334155
print(fib_tab(40))    # Output: 102334155
print(fib_opt(40))    # Output: 102334155

```

Example 2: 0/1 Knapsack

```
def knapsack(weights, values, capacity):
    """
    Classic 0/1 knapsack: maximize value without exceeding capacity.
    Time: O(n * capacity), Space: O(n * capacity)
    """
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            # Don't take item i
            dp[i][w] = dp[i-1][w]
            # Take item i (if it fits)
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])

    return dp[n][capacity]

weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 8
print(knapsack(weights, values, capacity)) # Output: 10 (items 2+4 or 1+3)
```

Example 3: Longest Common Subsequence (LCS)

```
def lcs(s1, s2):
    """
    Find length of Longest Common Subsequence.
    Time: O(m * n), Space: O(m * n)
    """
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]

print(lcs("ABCBDB", "BDCAB")) # Output: 4 (BCAB or BDAB)
print(lcs("AGGTAB", "GTXAYB")) # Output: 4 (GTAB)
```

Example 4: Coin Change Problem

```
def coin_change(coins, amount):
    """
    Minimum number of coins to make amount. Return -1 if impossible.
    Time: O(n * amount), Space: O(amount)
    """
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] = min(dp[a], dp[a - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1

print(coin_change([1, 5, 6, 9], 11)) # Output: 2 (5+6)
print(coin_change([2], 3))          # Output: -1 (impossible)
print(coin_change([1, 2, 5], 11))   # Output: 3 (5+5+1)
```

Interview Tips

- **Recognition:** If asked "is it possible to..." or "what is the minimum/maximum..." → think DP.
- **Steps:** 1) Define state, 2) Write recurrence, 3) Identify base cases, 4) Determine order.
- **Common Question:** "Climbing stairs" is essentially Fibonacci DP.
- **Optimization:** 2D DP tables can often be reduced to 1D or even two variables.

15. Common Interview Patterns

Concept Overview

Many coding interview problems follow recognizable patterns. Mastering these templates lets you solve new problems faster and more confidently.

Pattern 1: Prefix Sum

```

def subarray_sum_equals_k(nums, k):
    """
    Count subarrays whose sum equals k.
    Time: O(n), Space: O(n)
    """
    count = 0
    prefix_sum = 0
    seen = {0: 1} # prefix_sum -> frequency

    for num in nums:
        prefix_sum += num
        # If (prefix_sum - k) was seen before, those subarrays sum to k
        count += seen.get(prefix_sum - k, 0)
        seen[prefix_sum] = seen.get(prefix_sum, 0) + 1

    return count

print(subarray_sum_equals_k([1, 1, 1], 2))      # Output: 2
print(subarray_sum_equals_k([1, 2, 3], 3))      # Output: 2 ([1,2] and [3])
print(subarray_sum_equals_k([-1, -1, 1], 0))    # Output: 1

```

Pattern 2: Fast & Slow Pointers (Linked List)

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def has_cycle(head):
    """Floyd's cycle detection – O(n) time, O(1) space."""
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

def find_middle(head):
    """Find the middle node using slow/fast pointers."""
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

# Build list: 1 -> 2 -> 3 -> 4 -> 5
nodes = [ListNode(i) for i in range(1, 6)]
for i in range(4):
    nodes[i].next = nodes[i + 1]

print(find_middle(nodes[0]).val) # Output: 3

# Add a cycle: 5 -> 2
nodes[4].next = nodes[1]
print(has_cycle(nodes[0]))      # Output: True

```

Pattern 3: Monotonic Stack

```

def next_greater_element(nums):
    """
    For each element, find the next greater element to its right.
    Time: O(n), Space: O(n)
    Returns -1 if no greater element exists.
    """
    result = [-1] * len(nums)
    stack = [] # Stores indices

    for i, num in enumerate(nums):
        # Pop elements from stack whose value is less than current
        while stack and nums[stack[-1]] < num:
            idx = stack.pop()
            result[idx] = num
        stack.append(i)

    return result

print(next_greater_element([2, 1, 2, 4, 3]))
# Output: [4, 2, 4, -1, -1]

def largest_rectangle_histogram(heights):
    """
    Largest rectangle in histogram using monotonic stack.
    Time: O(n), Space: O(n)
    """
    stack = []
    max_area = 0
    heights = heights + [0] # Sentinel to flush stack

    for i, h in enumerate(heights):
        start = i
        while stack and stack[-1][1] > h:
            idx, height = stack.pop()
            max_area = max(max_area, height * (i - idx))
            start = idx
        stack.append((start, h))

    return max_area

print(largest_rectangle_histogram([2, 1, 5, 6, 2, 3])) # Output: 10

```

Pattern 4: BFS for Shortest Path

```

from collections import deque

def shortest_path(grid, start, end):
    """
    BFS for shortest path in unweighted grid.
    Time: O(rows * cols), Space: O(rows * cols)
    grid: 0 = passable, 1 = wall
    """
    rows, cols = len(grid), len(grid[0])
    queue = deque([(start, 0)]) # (position, distance)
    visited = {start}

    while queue:
        (r, c), dist = queue.popleft()
        if (r, c) == end:
            return dist
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                and grid[nr][nc] == 0
                and (nr, nc) not in visited):
                visited.add((nr, nc))
                queue.append(((nr, nc), dist + 1))
    return -1 # No path found

grid = [
    [0, 0, 0, 1],
    [1, 0, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 0]
]
print(shortest_path(grid, (0, 0), (3, 3))) # Output: 6

```

Pattern 5: Heap for Top-K Problems


```

import heapq

def top_k_frequent(nums, k):
    """
    Find k most frequent elements.
    Time: O(n log k), Space: O(n)
    """
    from collections import Counter
    freq = Counter(nums)
    # heapq is a min-heap; negate for max behavior
    return heapq.nlargest(k, freq.keys(), key=lambda x: freq[x])

def kth_largest(nums, k):
    """
    Find the kth largest element.
    Time: O(n log k), Space: O(k)
    """
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]

print(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) # Output: [1, 2]
print(kth_largest([3, 2, 1, 5, 6, 4], 2))   # Output: 5

```

Interview Tips

- **Pattern Recognition Chart:**

- Subarray sum / contiguous range → Prefix sum + hash map
- Linked list cycle / middle → Fast & slow pointers
- Next greater/smaller element → Monotonic stack
- Shortest unweighted path → BFS
- Shortest weighted path → Dijkstra (heapq)
- Top K / K closest → Min-heap of size K
- Sorted subsets / permutations → Backtracking
- Overlapping subproblems → Dynamic programming

- **Practice Order:** Two Sum → Binary Search → BFS/DFS → Sliding Window → DP

- **Time Management:** Spend 5 minutes identifying the pattern before coding.

End of Python Interview & Coding Study Guide

Last updated: 2025 | All code tested with Python 3.10+